

The Seamless Shopping Experience

Assignment #4

Software Systems Architecture

1MEIC02 T3

Gonçalo Gonçalves da Costa Pinto
José Diogo Alves Granja
Manuel Mo
Pedro Rafael Correia Borges
Ricardo Miguel Yang

1. System Overview: The Seamless Shopping Experience

Our system revolutionizes physical retail by removing the friction of traditional checkout lines. By integrating biometric identification and Radio-Frequency Identification (RFID) technology, we bridge the gap between online convenience and in-store shopping. The core of this experience is a fully automated journey—from the moment a customer picks up a cart to the moment they walk out the door.

1.1. Registered Users: The Fully Automated Journey

For customers using our app, the experience is completely **GRAB-AND-GO**, following the procedure shown in Figure 1 and the instructions below:

1. **Initial Setup:** Users register on the app by providing a facial recognition scan and linking a preferred payment method.
2. **The Smart Entry:** When a customer enters the store and picks up a cart or basket (equipped with a chip and a built-in screen), they pass through an entry gateway. This gateway uses facial recognition to identify the user and automatically links them to that specific cart.
3. **Smart Shopping:** As items are placed in the cart, RFID sensors detect them instantly. The running list and total are displayed on both the cart's screen and the user's phone. The system automatically applies personalized coupons and member discounts, when available.
4. **Interactive Features:** The cart provides smart suggestions based on what is currently in the basket or what is trending. Users can also search for items by name; the system will then display a store map with directions to the product's exact location.
5. **Seamless Checkout:** At the exit gateway, a large display shows the final shopping list. The system performs a final facial recognition check. If the person pushing the cart has changed since entry, the system recalculates discounts for the new user (the gateway detects the cart by RFID sensors). Once the customer confirms the list via facial recognition, the payment is processed automatically, and the gate.

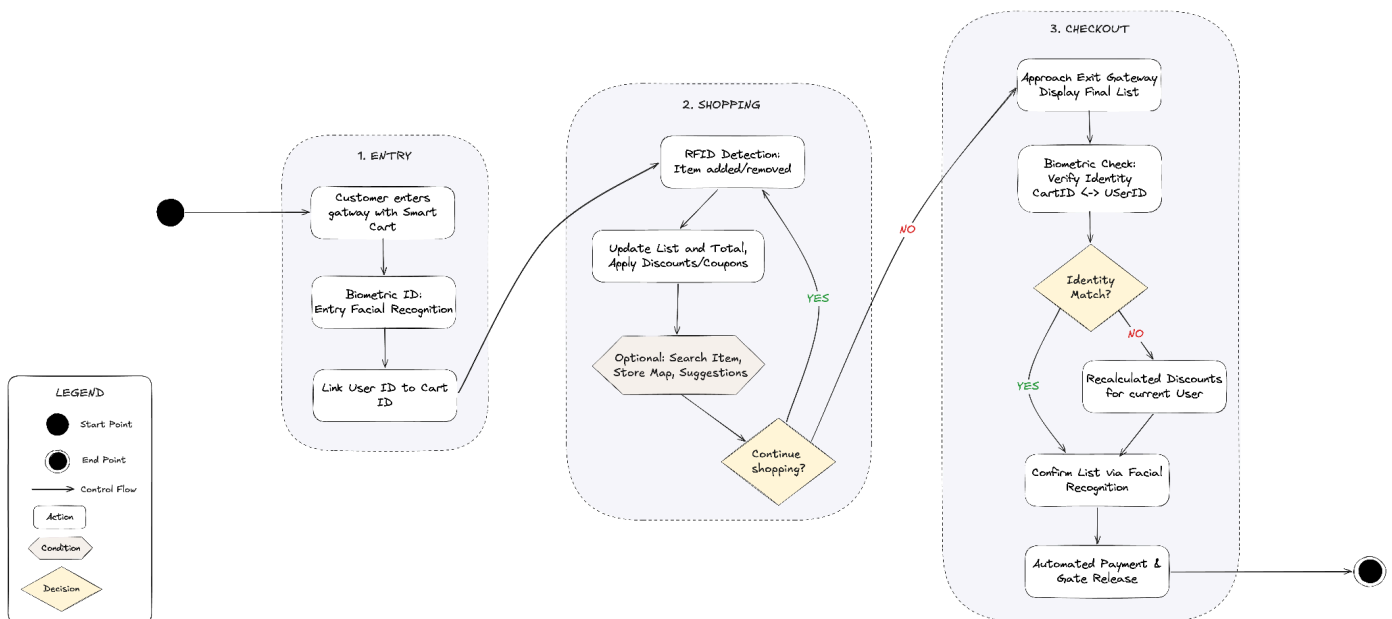


Figure 1: UML Activity Diagram depicting the automated “grab-and-go” journey for registered users.

1.2. Guest Shopping: Convenience Without an Account

We ensure that customers without the app or an account still enjoy an engaging experience. The process is very similar for registered customers, though with a few differences, summarized in Table 1, and listed below:

1. **Identification:** Upon entry, the gateway uses facial recognition to identify the individual for security and tracking purposes, though no personal account is linked.
2. **Shopping Process:** Guests use the same smart carts. While they see the running total and product information on the cart's screen, they do not receive personalized coupons or automated member discounts.
3. **Manual Payment:** At the exit gateway, the shopping list is displayed on the large screen just like with registered users. However, instead of an automatic charge, the customer completes the transaction using a payment terminal (for credit/debit cards) located right on the cart or basket display.

Table 1: Comparison of System Features and User Journeys for Registered vs. Guest Shoppers.

Feature	Registered Users	Guest Shoppers
Setup	Pre-register facial recognition and payment method defined on app	No registration needed
Entry Association	Connects the face to the user account, and associates the cart or basket	Connects the face to a temporary shopping session and to the cart
Pricing	Personalized (member deals and coupons)	Standard (shelf pricing)
Cart Sync	Updates cart items on app and cart screen	Updates cart items on cart screen only
Checkout	Confirms identity for auto-payment	Matches person to cart for manual payment at the terminal
Gate Release	Opens upon biometric confirmation	Opens upon terminal approval

1.3. System Architecture Highlights

- **Biometric Security:** We rely strictly on facial recognition for both identification and purchase confirmation, eliminating the need for physical security measure entry.
- **Real-Time Binding:** The system creates a temporary “link” between the human biometric data, the cart's hardware ID, and the items detected by RFID.
- **Dynamic Pricing:** Prices and discounts are calculated in real-time, ensuring that the total reflects the identity of the person currently in possession of the cart at the moment of checkout.

1.4. Admin Control and Store Management

- **Product and Price Management:** Admins maintain the central database, setting base prices, scheduling sales, and managing the electronic coupons that are automatically applied to registered users' carts.
- **Store Mapping:** They manage the digital store layout, ensuring the “item lookup” feature provides customers with accurate directions and aisle locations for every product.
- **Security Oversight:** The system provides real-time alerts if a store alarm is triggered, allowing admins to respond immediately if a customer exits without completing the facial recognition confirmation.
- **Hardware Monitoring:** Admins track the operational status of all hardware, including the smart cart screens, RFID sensors, and entry/exit gateways to ensure a smooth shopping experience.
- **Inventory Integration:** They oversee the flow of goods by monitoring automated stock levels, as the system tracks items being added or removed from carts in real-time.

2. Main Quality Attribute Requirements

The success of our system is measured not only by its functional capabilities, but also on how well it handles real-world physical retail constraints. For example, a slight delay in an app might be acceptable at home, but delays at the physical store exit create a crucial bottleneck. Taking this into account, the architecture is driven by the following key quality attributes:

- **Performance (real-time responsiveness):** When a customer adds an item into the cart, the system must detect it, calculate the applicable discounts, and update the running total on the cart's screen and user's phone app almost instantly.
- **Availability (fault tolerance during shopping and checkout):** The core shopping and checkout experience must have near 100% uptime, meaning that if a secondary system (like the recommendation engine) crashes, the customers should still be able to add items and pay. This is critical since a system failure at checkout would either physically trap the customer in the store or force them to leave without their groceries.
- **Security (authentication, payment and loss prevention):** The system must secure user payment data, ensure that the person walking out of the store is the authorized user (preventing cart and account theft), and trigger physical store alarms if an unconfirmed exit occurs.
- **Modifiability (extensibility for future business rules):** Since the retail business is highly dynamic, the store admin must be able to add new pricing rules, promotional campaigns, or integrate new payment methods without requiring a complete system overhaul that would lead to significant downtime.
- **Usability (seamless interaction across interfaces):** The system experience must be intuitive and seamless for both mobile app users and guests relying only on the cart-mounted device, with the information displayed remaining consistent across both interfaces at all times to avoid confusing the customers.

To ensure that these quality attributes are met, we mapped specific architectural patterns to solve each requirement (shown in Table 2). The detailed explanation of these patterns and how they are applied will be further explored in Section 4.

Table 2: Mapping of Quality Attributes to Architectural Patterns.

Quality Attribute	Primary Pattern(s) Addressing It	How it Solves the Problem
Performance	Event-Driven, Pipes and Filters	Asynchronous processing (event driven architecture) prevents the cart UI from freezing while waiting for the backend. Pipes and Filters allow complex pricing and discount rules to be processed efficiently in parallel.
Availability	Microservices	Fault isolation guarantees that if a non-critical service (e.g., Recommendations) crashes, it does not take down the checkout or cart total systems.
Security	API Gateway	Acts as a centralized checkpoint, ensuring all requests (mobile app, cart, or admin) are authenticated, and biometric validation is completed before external payment processing.
Modifiability	Pipes and Filters, Microservices	New discount rules can be dropped into the pricing pipeline as independent filters, and new payment methods can be added as isolated microservices without touching the core system.
Usability	MVC (Model-View-Controller)	Separating the backend Model from the UI Views ensures the customer sees the exact same synchronized data whether they look at their phone or the cart screen.

3. C4 Model

In order to better understand our philosophy, the architecture of the Automated Shopping System is documented using the C4 model, a hierarchical approach that progressively reveals system detail across four levels of abstraction. The three levels presented below move from a high-level context view down to the individual components that make up each container.

3.1. Level 1 - System Context

The diagram, in Figure 2, places the **Automated Shopping System** within its broader environment, identifying the three human actors and two external systems that interact with it. The Registered Customer identifies via facial recognition, shops using the smart cart, and confirms their purchase through biometric validation at the exit gateway. The Guest Customer follows a similar path, shopping with the smart cart and identifying via facial recognition, but completes the transaction manually through the payment terminal. The Store Admin interacts with the system to set prices, manage promotional coupons, update store maps, and monitor real-time alerts.

Below the system boundary, two external systems integrate with the platform. The Payment Provider handles all financial transactions, receiving payment requests and returning status confirmations for credit/debit card and MBWay operations. The Inventory Management System serves as the authoritative source for product details, stock levels, and store layout data, with the Automated Shopping System fetching and syncing this information in real time.

At the centre sits the Automated Shopping System itself: the core platform responsible for biometric linking, RFID item tracking, dynamic pricing calculation, and physical gateway control.

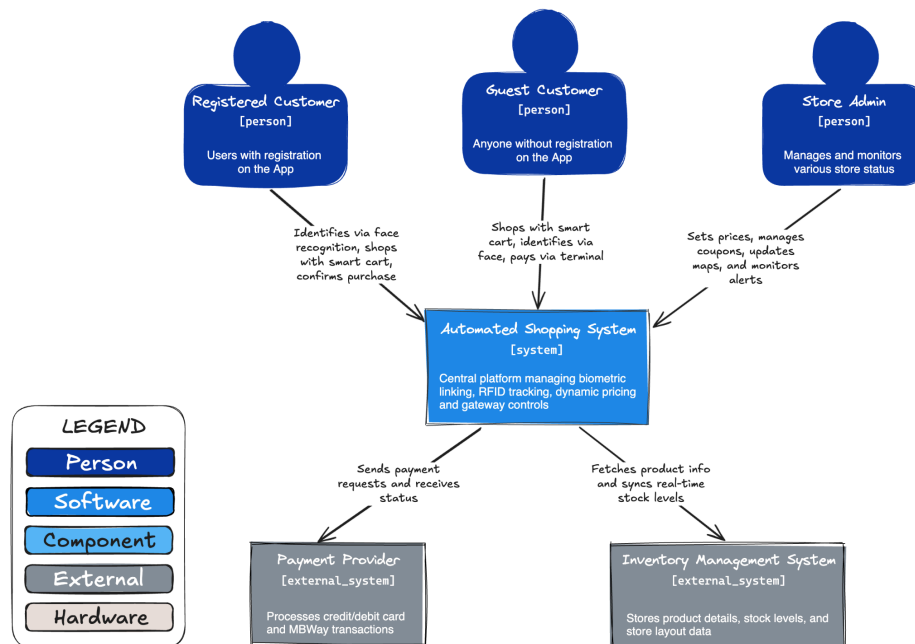


Figure 2: C4 Level 1 Diagram.

3.2. Level 2 - Container Diagram

The diagram below (Figure 3) zooms into the Automated Shopping System to reveal its **five internal containers** and how each actor interacts with them. The Guest Customer interacts exclusively with the Smart Cart: a hardware-backed container that detects products via RFID and displays the running list on the cart's terminal. The Registered Customer interacts with the system through the Mobile App, which shows the current cart contents synced in real time. The Store Admin manages the store through the Admin Panel, which provides visibility over inventory, pricing, and store configuration.

The Smart Cart communicates with the **Store Physical System** via RFID calls to forward hardware events such as item additions and biometric scans. Additionally, since the cart is equipped with a built-in terminal that displays product lists and handles guest payments, it also communicates with the Cloud Services container directly (likely via HTTP/JSON API calls) to keep the cart screen updated in real time. The Store Physical System is the local container responsible for handling all physical hardware events, and it forwards structured data to the Cloud Services container via API calls. Both the Mobile App and the Admin Panel communicate directly with Cloud Services through standard HTTP/JSON API calls, making this container the

central hub of backend logic: responsible for providing smart shopping functionality including cart session management, dynamic pricing, and payment processing.

Cloud Services reads from and writes to the Database, which persists all system state. The database content is noted as still to be defined in the diagram. Cloud Services also communicates outward to the Payment Provider external system, sending payment requests and receiving transaction status in return.

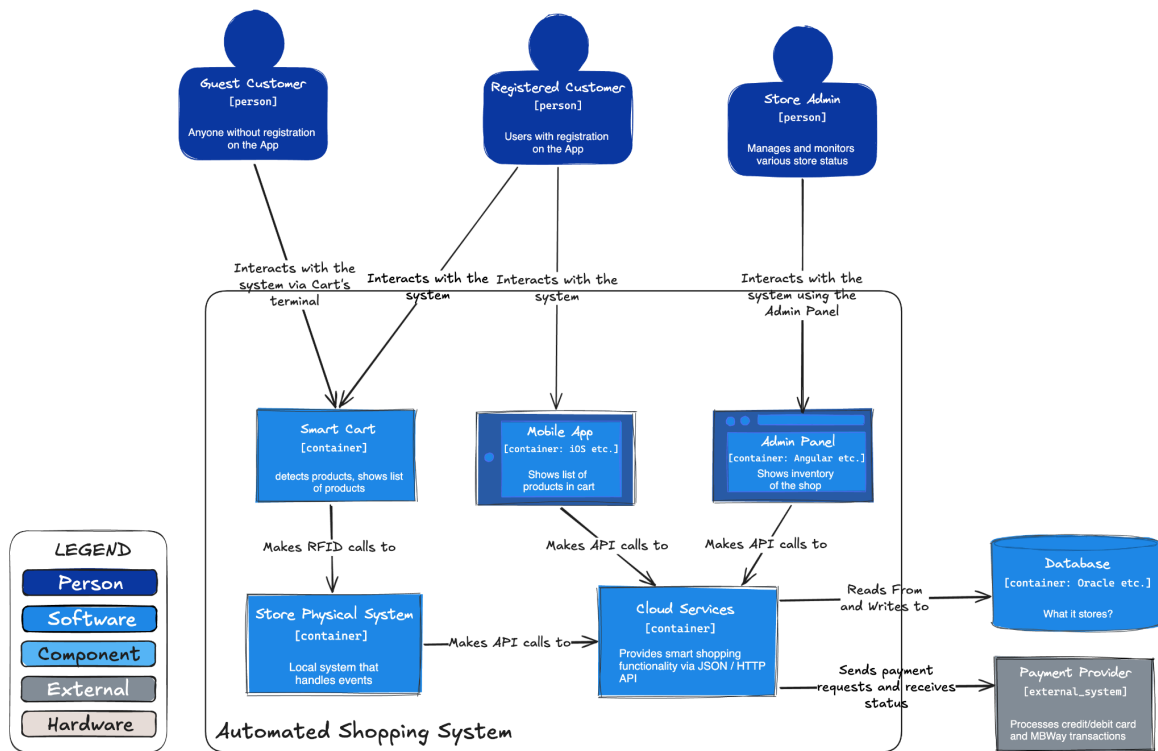


Figure 3: C4 Level 2 Diagram.

3.3. Level 3 - Component Diagram

The Level 3 C4 diagram (Figure 4) provides a deeper view into the internal structure of the Cloud Services and Store Physical System containers, breaking them down into their core components and illustrating how responsibilities are distributed across the system.

At the heart of the backend architecture lies the **API Gateway**, which acts as the single entry point for all external interactions. It validates incoming requests from the Mobile App, Admin Panel, and Smart Cart terminal, and routes them to the appropriate internal services. This ensures a clean separation between client-facing interfaces and business logic, while also centralizing concerns such as authentication, throttling, and request validation.

The **Cart Session Manager** is responsible for maintaining the association between users and their shopping carts. It processes cart-related events received from the API Gateway and keeps track of cart contents in real time. This component also orchestrates interactions with other services to compute the cart’s state, including requesting pricing calculations and retrieving inventory data.

Pricing logic is handled by the **Dynamic Pricing Engine**, which calculates the total cost of the cart by applying pricing rules, discounts, and promotional coupons. This component ensures that pricing remains flexible and can adapt dynamically to different business strategies, such as member discounts or time-based promotions. This is useful since a list can be divided into individual items even if they are “thrown” together to the cart, ensuring that each item is evaluated independently while still contributing to the overall cart total.

To enhance the shopping experience, the **Recommendation Service** analyzes cart contents and generates product suggestions. By leveraging data such as item associations and promotional trends, it provides personalized recommendations that can be displayed on the cart terminal or mobile application.

User-related operations are encapsulated within the **User Service**, which manages customer profiles, payment methods, and coupon data. It supports both retrieving and updating user information, enabling a personalized and seamless checkout experience.

The **Inventory Service** is responsible for maintaining product availability and location data. It tracks stock levels and provides information about where items are located within the store (e.g., aisle and shelf). This service interacts directly with the database and ensures that all components operate on consistent and up-to-date inventory data.

Payment processing is handled through the **Payment Interface**, which provides a simplified abstraction over external payment systems. It enables one-click checkout functionality and communicates with the external Payment Provider to process transactions and receive confirmations.

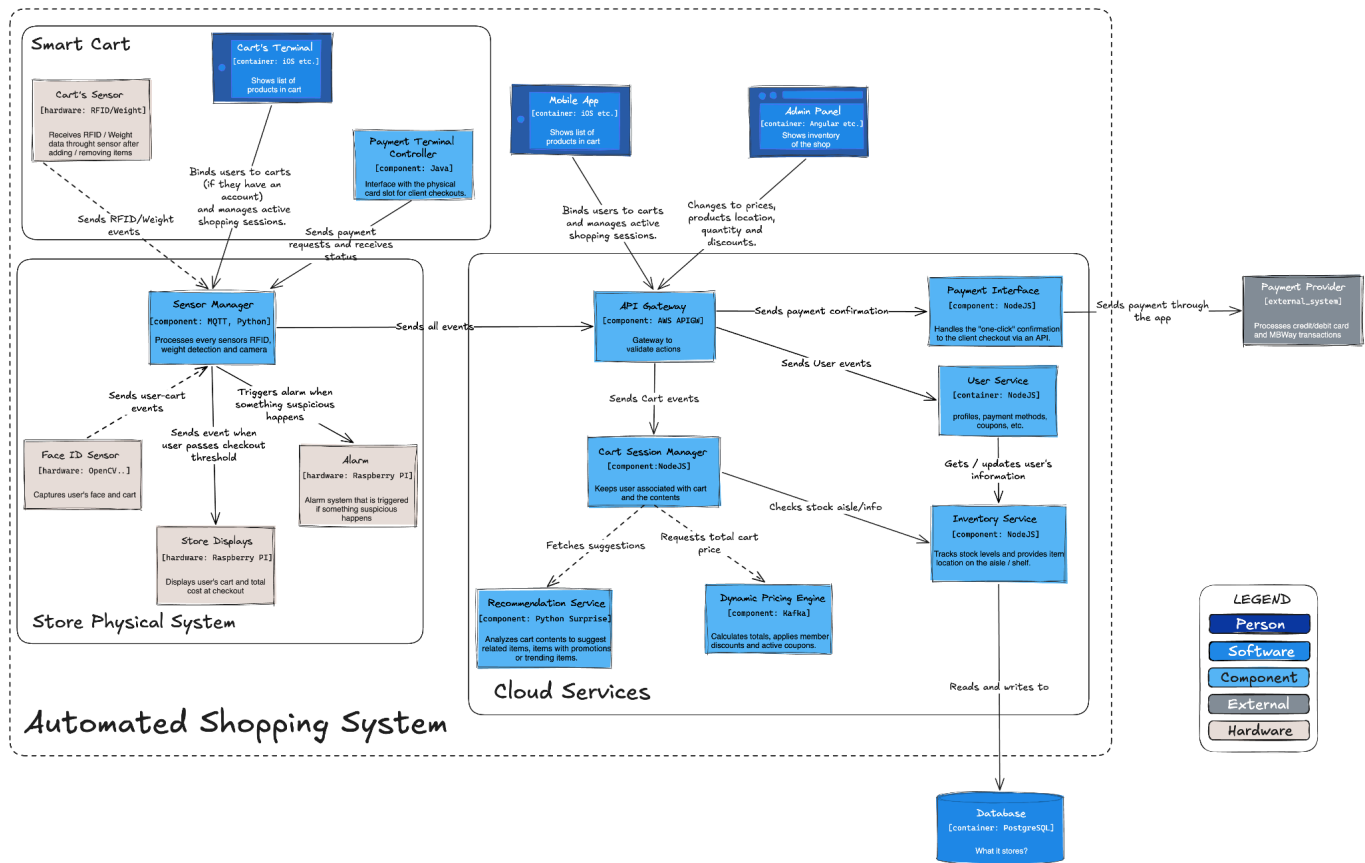
All persistent data is stored in the **Database**, which serves as the single source of truth for the system. It maintains information related to users, products, cart sessions, transactions, and inventory.

On the physical side, the **Sensor Manager** within the Store Physical System processes all incoming hardware events, including RFID scans, weight detection, and camera inputs. It translates raw sensor data into structured events and forwards them to the Cloud Services via the API Gateway. Additionally, it monitors for suspicious activity and can trigger the **Alarm** component when necessary.

Supporting components such as the **Face ID Sensor** and **Store Displays** integrate with the Sensor Manager to capture biometric data and present checkout information to users. These components ensure seamless interaction between the physical shopping environment and the digital backend.

It is important to note that this level of the diagram intentionally **abstracts away lower-level implementation details**, such as internal controllers, UI logic, and fine-grained class structures. Including such elements would significantly increase the complexity of the diagram and reduce its readability. Instead, the focus is placed on the major functional components and their interactions, providing a balance between clarity and architectural insight.

Overall, this level of the diagram highlights how the system decomposes into specialized, loosely coupled components that collaborate through well-defined interfaces. This modular approach improves scalability, maintainability, and flexibility, allowing the system to evolve as new features and requirements emerge.



4. Architecture Patterns Used

Designing a hybrid system that bridges physical retail hardware (such as smart carts and RFID sensors) with cloud-based backend services requires a robust and flexible foundation. To satisfy the Quality Attributes defined earlier (such as real-time performance, high availability, and strict security), our architecture relies on a combination of proven design patterns, which are detailed in the following subsections.

4.1. API Gateway

The API Gateway pattern establishes a single entry point for all client-facing communication with the backend. In the Smart Shopping Cart system, the Mobile App and the Admin Panel interact exclusively with the API Gateway, never directly with internal services. There are some reasons why we decided to use this architectural pattern:

- **Security and Access Control:** Without a gateway, every internal service would need to implement its own authentication and authorization logic. By centralizing this in the API Gateway, we ensure that only authenticated users can reach internal services. For example, an admin trying to update product prices is validated once at the gateway — and identified as an admin role — before the request is forwarded to the Inventory Service. A regular customer using the Mobile App would be blocked from ever reaching that service.
- **Client Simplification:** The Mobile App only needs to know one address — the API Gateway. If the Cart Session Manager moves or changes its internal API, the client is completely unaffected. Only the gateway's routing configuration needs to be updated, which greatly improves maintainability.
- **Cross-Cutting Concerns:** The gateway is the natural place to handle concerns that apply to every request, such as rate limiting, logging, and load balancing, avoiding duplication of this logic across every individual service.

4.2. Microservices

Rather than building the backend as a single monolithic application, the Smart Shopping Cart system is decomposed into a set of small, independently deployable services, each with a single well-defined responsibility. The Cloud Services layer contains the Cart Session Manager, Dynamic Pricing Engine, Recommendation Service, Inventory Service, User Service, and Payment Interface — each running independently. There are a couple of reasons why we decided to use this pattern:

- **Independent Scalability:** During peak shopping hours, the Recommendation Service may receive significantly more load than the Inventory Service. With microservices, we can scale only the services that need it, without scaling the entire backend, reducing infrastructure costs.
- **Fault Isolation:** If the Recommendation Service crashes, the rest of the system continues to function normally. The customer can still add items, checkout, and pay. In a monolithic system, a bug in the recommendation logic could bring down the entire application.
- **Independent Development and Deployment:** Each service can be developed, tested, and deployed independently by different team members. For example, the team working on the Dynamic Pricing Engine does not need to coordinate deployments with the team working on the Inventory Service.

4.3. Event-Driven Architecture (EDA)

The Smart Shopping Cart system is fundamentally reactive. Instead of the backend having to constantly “ask” the cart if anything has changed, the system relies on an asynchronous choreography where components react to state changes in real-time. There are a couple of reasons to why we decided to use this architectural pattern:

- **Decoupling of Producers and Consumers:** The Sensor Hardware does not need to know that the Recommendation Service or the Inventory System even exist. It simply broadcasts an `ITEM_ADDED` event. This makes it possible to add new features, like a feature that illuminates the aisle a person is in without changing the code of the cart.
- **Real-time Responsiveness:** If we used a traditional req-rep model, if the Pricing Engine was busy, the cart screen might wait for a total. Using an EDA, the event is fired into a message queue. The cart UI can immediately show for example an “Updating...” state while the Engine processes the event in the background.

- **High Scalability and Throughput:** On a certain day, the store might have a lot of carts firing events simultaneously. EDA allows the Store Hub to buffer these spikes in a message queue like MQTT or Kafka. The backend services can then process the events without crashing under a sudden surge of HTTP requests.

4.4. Pipes and Filters

While our system is globally event-driven, we use the Pipes and Filters pattern within the pricing Engine and the Sensor Manager to handle complex data transformations. This pattern allows us to treat data processing as a stream, where discrete Filters perform specific tasks on the data before passing it through Pipes.

The pricing pipeline is something like this:

1. **Validation Filter:** Receives the raw RFID event and checks if the item ID exists in the Inventory Service.
2. **Base Price Filter:** Fetches the standard shelf price from the database.
3. **Promotion Filter:** Checks the “Active Sale Rules” provided by the Admin Panel.
4. **Member Discount Filter:** If a user is bound to a cart, this filter checks their tier and applies personalized coupons (and electronic coupons)
5. **Tax/VAT Filter:** Calculates the final tax based on the store's location.
6. **Total Accumulator:** Aggregates results into a final running total.

Why this pattern?

- **Modular Extensibility:** If the store adds a new system for discounts, they don't have to rewrite the whole Pricing Engine. We can simply drop a new filter into the pipeline.
- **Testing and Debugging:** Because each filter is a unit in itself, we can test one filter independently. This reduces the risk of “spaghetti code”.
- **Parallel Processing:** If we have a massive cart with for example 30 items, different filters can potentially run in parallel or be optimized to handle the stream without lag, ensuring **Performance**.

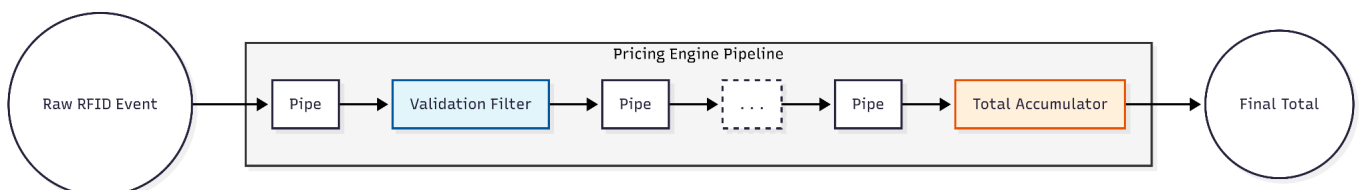


Figure 5: Pipes and Filters Pattern of the system.

4.5. Model-View-Controller (MVC)

The MVC pattern separates an application into three interconnected components, dividing the internal representation of data from the way it is presented and accepted by the user. The reasoning behind our choice for this pattern is:

- **Separation of concerns:** display logic is completely separated from business logic, making each layer independently testable and maintainable.
- **Consistency across clients:** both the Mobile App and the Cart Terminal share the same backend Model (Store Backend Services), but have completely different Views, which is only possible because of this separation.
- **Maintainability:** changing the UI of the Mobile App does not affect the underlying cart logic at all.

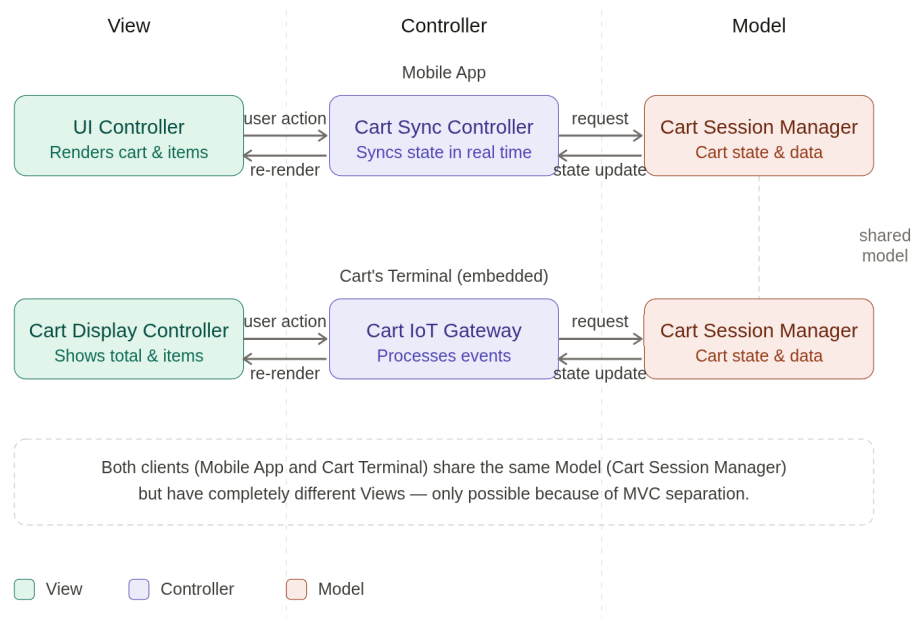


Figure 6: MVC Pattern of the system.

4.6. Other Patterns

Beyond the primary patterns described above, the architecture also makes use of several supporting patterns that contribute to the overall design:

- **Hub-and-Spoke:** used within the Store Physical System, where the Sensor Manager acts as the central hub coordinating all physical hardware. The Cart's Sensor, Face ID Sensor, and Alarm connect as spokes, centralizing all hardware event processing before forwarding structured events to the Cloud Services.
- **Observer:** used in conjunction with MVC. The Mobile App and Cart's Terminal act as observers that automatically update their displays whenever the Cart Session Manager broadcasts a change in the total price or item list.
- **Repository:** used by the microservices to abstract database interactions. Services like Inventory Service and User Service fetch and write data without exposing any underlying SQL or NoSQL query specifics to the business logic layer.
- **Facade:** used for external payment integration. The Payment Interface component provides a simplified interface for the system to interact with complex third-party APIs like MBWay, Visa, and Mastercard, hiding their complexity from the core checkout logic.

6. Excerpts of Code

This section presents pseudocode for some logic of the system, focusing on how the shopping and checkout processes work. In Figure 7, it is introduced the cart and item data structures that model a part of the system's internal state. These structures establish the foundation upon which the remaining logic operates.

Building on these definitions, Figure 8 illustrates the process of adding an item to the cart. It details how RFID signals captured by the cart sensors are interpreted and mapped to product identifiers, which are then validated with the shopping inventory. The function also demonstrates how the system handles duplicate products by incrementing the quantity of an existing item rather than creating a new entry, ensuring both efficiency and consistency in the cart state.

Finally, Figure 9 presents the checkout process, highlighting the interaction between biometric authentication and payment handling. In this step, facial recognition is used to verify the identity of the user associated with the cart before confirming the transaction. Once validated, the system proceeds with payment processing and finalizes the session, completing the seamless "grab-and-go" experience.

Together, these excerpts provide a concrete view of how the system behaves at runtime, bridging the concepts described throughout this document with their practical implementation.

```
1 OBJECT Product:
2   id   : String
3   name : String
4   price : Float
5   discount : Float
6
7 OBJECT CartItem:
8   product : Product
9   quantity : Integer
10
11 OBJECT Cart:
12   cartId : String
13   userId : String
14   items : List<CartItem>
15   status : String      # "active" | "checked_out" | "checking_out"
16   scanner : Scanner
17   priceWithDiscounts : Float
```

Figure 7: Pseudocode of the cart and item objects

```
1 FUNCTION scanAndAdd(cart):
2   signal = cart.scanner.read()
3
4   IF signal is empty:
5     RETURN
6
7   productId = extractId(signal)
8   product = database.lookup(productId)
9
10  IF product NOT FOUND:
11    RETURN
12
13  existingItem = findInCart(cart.items, product.id)
14
15  IF existingItem EXISTS:
16    existingItem.quantity += 1
17  ELSE:
18    newItem = CartItem {
19      product: product.id,
20      quantity: 1,
21    }
22    cart.items.append(newItem)
23
24  cart.priceWithDiscounts = calculatePrice(cart) # These will calculate the current price of the cart with the product and user discounts
```

Figure 8: Pseudocode of an item being added to the cart

```
1 FUNCTION checkout(cart):
2   IF cart.status != "active":
3     call_security()
4     RETURN
5
6   cart.status = "checking_out"
7   faceData = camera.capture()
8   verifiedUser = facialRecognition.identify(faceData) # Can be a NULL ID that means the user does not have an account
9
10  IF verifiedUser.id != cart.userId:
11    cart.userId = verifiedUser.id
12    cart.priceWithDiscounts = calculatePrice(cart)
13    RETURN
14
15  user = database.getUser(cart.userId)
16  processPayment(user)
17  generateReceipt(cart)
18  cart.status = "checked_out"
```

Figure 9: Pseudocode of the checkout process

7. Conclusion and Future Work

The Smart Shopping Cart system brings together biometric identification, RFID sensors, and a cloud-connected backend to close the gap between online and physical retail, removing the checkout line without removing the familiarity of store shopping.

The patterns chosen throughout the architecture are not decorative; they each pull their weight. Together they tackle real-time responsiveness, fault tolerance, security, and extensibility in a way that a simpler monolithic design simply could not. The dual-interface approach, phone app and cart terminal, means no customer is left out, and the careful attention to when things bind together (cart pairing, pricing, payment) keeps the system flexible enough to handle the messiness of real-world retail.

The architecture is solid enough to serve the store in the present and open enough to grow with it in the future. Looking forward, there are several natural extensions that can further enhance both usability and system capability. Supporting multi-user cart sessions is one of the most relevant improvements, enabling scenarios such as families shopping together without triggering identity conflicts. This could evolve into shared sessions across multiple registered users, allowing automatic bill splitting or collaborative shopping experiences.

Another important scenario to address is handling cases where a customer abandons a cart or leaves the store without completing the checkout process. Future improvements could include automatic session timeout and cart release mechanisms, re-association of carts to new users, or alert systems that distinguish between intentional abandonment and potential fraud. This would require refining the cart-to-user binding lifecycle and introducing clearer state management for inactive or orphaned sessions.

From a systems perspective, future work could also explore stronger resilience mechanisms, such as offline capabilities for users logged into a cart in case of temporary connectivity loss, as well as the use of edge computing at the store level to reduce latency and dependency on cloud services. Additionally, privacy and ethical considerations around biometric data usage could be further strengthened by incorporating features such as user-controlled data visibility, anonymization layers, or alternative authentication methods.

Overall, the architecture is solid enough to serve current retail needs while remaining open to evolution. Its modular, pattern-driven design ensures that future requirements, whether functional, technical, or regulatory, can be incorporated incrementally without requiring a full system redesign.

8. References

1. Polacco, A., & Backes, K. (2018). The amazon go concept: Implications, applications, and sustainability. *Journal of Business and Management*, 24(1), 79-92.
2. Wikipedia contributors. (n.d.). *Amazon Go*.
https://en.wikipedia.org/wiki/Amazon_Go
3. Group C2T3 of ASSO (2026). C4 and UML architectural activity diagrams. Excalidraw.
https://excalidraw.com/#room=ddf2737bf30d088330b2,bYw9Nkzkzx0PLP_5fEly2Q
4. Solo.io. (n.d.). *API gateway pattern*.
<https://www.solo.io/topics/api-gateway/api-gateway-pattern>
5. Amazon Web Services. (n.d.). *Event-driven architecture*.
<https://aws.amazon.com/pt/event-driven-architecture/>
6. GeeksforGeeks. (n.d.). *Repository design pattern*.
<https://www.geeksforgeeks.org/system-design/repository-design-pattern/>
7. Evidi. (2025). *Hub-and-spoke architecture*.
<https://www.evidi.com/techdictionary/hub-and-spoke-architecture>